MACHINE LEARNING · STUDY GUIDE

Decision Tree Analysis

Classification and regression trees from first principles to Random Forest, XGBoost, LightGBM and 20+ tree models, explained with executed Python code and walkthroughs.



PREPARED BY

Chandan Chaudhari

GitHub: github.com/chandanc5525

Contents

1. Introduction 4

1.1 Anatomy of a tree 4

1.2 Classification vs regression trees 4

1.3 Strengths and weaknesses 5

1.4 The tree family at a glance 5

1.5 Python toolkit 5

2. Intuition 6

2.1 The idea in four steps 6

2.2 Three mental pictures 6

3. Mathematics 7

3.1 Impurity measures for classification 7

3.2 Information gain of a split 8

3.3 Other classification criteria 8

3.4 Regression trees: variance reduction 8

3.5 The training algorithm (CART) 9

3.6 Stopping rules and pruning 9

3.7 Feature importance 9

4. Evaluation Metrics 11

4.1 Classification trees 11

4.2 Regression trees 11

4.3 Tree-specific diagnostics 12

4.4 Computing classification metrics in Python 12

5. Use Cases 13

5.1 When to use a tree 13

5.2 When to avoid or upgrade 14

6. Python Code Structure for Tree Projects 14

6.1 The estimator API for trees 14

6.2 Key hyper-parameters 15

6.3 Inspecting the tree structure 15

6.4 Preprocessing: categorical features and missing values 16

6.5 Recommended project workflow 17

6.6 A reusable evaluation helper 17

7. Classification Trees 18

7.1 Reading the tree as IF-THEN rules . . . 18
7.2 Splitting criteria . . . 19
7.3 Overfitting and the depth limit . . . 19
7.4 Tuning and cost-complexity pruning . . . 20
7.5 Feature importance . . . 21
7.6 Imbalanced classes . . . 21

8. Regression Trees . . . 22

8.1 A staircase fit . . . 22
8.2 Regression metrics on real data . . . 23
8.3 Criteria, tuning and the extrapolation limit . . . 24
8.4 Multi-output regression trees . . . 25

9. More Tree Models: Ensembles and Variants . . . 25

9.1 Decision stump and AdaBoost . . . 25
9.2 Bagging and Random Forest . . . 26
9.3 Extremely Randomized Trees (Extra Trees) . . . 27
9.4 Gradient Boosting . . . 28
9.5 HistGradientBoosting . . . 29
9.6 XGBoost and LightGBM . . . 29
9.7 Voting and Stacking ensembles . . . 30
9.8 Isolation Forest (anomaly detection) . . . 31
9.9 Random Trees Embedding: trees as feature generators . . . 31
9.10 Other tree algorithms and variants . . . 32

10. Model Comparison and How to Choose . . . 33

10.1 Classification results . . . 33
10.2 Regression results . . . 34
10.3 Decision guide . . . 35

11. Cheat Sheet and Summary . . . 35

Learning roadmap . . . 36

Document map

Sections 1–6 cover introduction, intuition, mathematics, evaluation metrics, use cases and the Python code structure. Sections 7 and 8 explain classification and regression trees. Section 9 covers more than twenty further tree models: stumps, AdaBoost, bagging, Random Forest, Extra Trees, Gradient Boosting, HistGradientBoosting, XGBoost, LightGBM, voting, stacking, Isolation Forest and more. Section 10 compares them on real data.

1. Introduction

A **Decision Tree** is a **supervised learning** algorithm that predicts a target by asking a sequence of simple **yes/no questions** about the features. The questions form a tree: every internal node tests one feature, every branch is an answer, and every **leaf** holds the final prediction. The same idea solves **classification** (predict a class) and **regression** (predict a number).

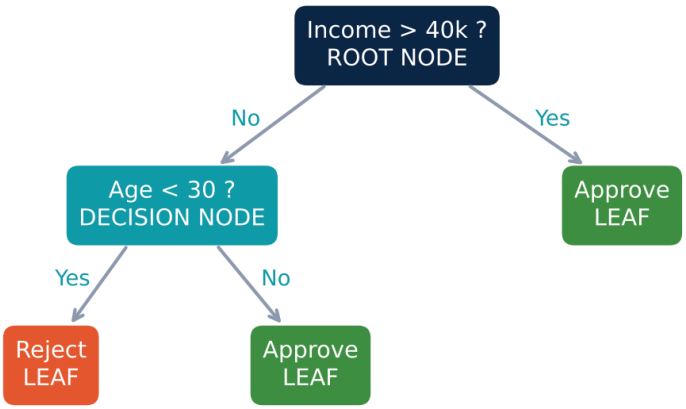


Figure 1. A tiny decision tree for loan approval: root, decision node and leaves.

1.1 Anatomy of a tree

Term	Meaning
Root node	The first question; contains the whole training set
Decision (internal) node	A test on one feature, e.g. Age < 30, that splits the data into child nodes
Branch	One possible answer to a test (Yes / No)
Leaf (terminal) node	Final prediction: a class (classification) or a number (regression)
Depth	Length of the longest path from the root to a leaf
Pruning	Removing weak branches to reduce overfitting
Impurity	How mixed the classes (or how spread the values) are inside a node

1.2 Classification vs regression trees

	Classification tree	Regression tree
Target	Category (spam / not spam)	Number (house price)
Split quality	Gini impurity or entropy	Squared error (variance)
Leaf prediction	Majority class, plus class fractions as probabilities	Mean of the training targets in the leaf
scikit-learn class	DecisionTreeClassifier	DecisionTreeRegressor
Typical metrics	Accuracy, F1, ROC-AUC	MAE, RMSE, R^2

1.3 Strengths and weaknesses

Strengths	Weaknesses
Readable like a flow chart; can be exported as IF-THEN rules	A single deep tree overfits and is unstable (high variance)
No feature scaling needed	Cannot extrapolate beyond the training range (regression)
Handles numeric and categorical features, non-linear effects and interactions	Axis-parallel cuts need many splits for diagonal boundaries
Fast prediction, robust to outliers in features	Greedy search is not globally optimal; biased to many-valued features
Building block of Random Forest and Gradient Boosting	Small data changes can produce a very different tree

1.4 The tree family at a glance

Group	Models covered in these notes
Classic single trees	ID3, C4.5/C5.0, CART, CHAID, decision stump (Sections 3, 7, 8)
Averaging ensembles	Bagging, Random Forest, Extra Trees (Section 9)
Boosting ensembles	AdaBoost, Gradient Boosting, HistGradientBoosting, XGBoost, LightGBM (Section 9)
Combined ensembles	Voting, Stacking (Section 9)
Special purpose	Isolation Forest, tree-based embeddings, quantile and monotonic boosting (Section 9)

1.5 Python toolkit

Library	Role
NumPy	Arrays; used to build a tree from scratch
pandas	Tabular data and readable importance tables
scikit-learn	Trees, ensembles, pipelines, cross-validation, metrics, inspection
XGBoost / LightGBM	High-performance gradient-boosted trees
Matplotlib	Tree plots and decision boundaries

> **How to read the code in this document**
>
> Every Python block is **executed for real** while this PDF is generated, so each **OUTPUT** panel is genuine. Blocks share one session in order and are followed by a **Code walkthrough** table. Install with `pip install numpy pandas scikit-learn xgboost lightgbm matplotlib`.

2. Intuition

Think of the game **20 Questions**. To guess an animal quickly you first ask the question that **splits the possibilities most evenly** ("Is it a mammal?"), then keep asking follow-ups until you are certain. A decision tree learns the best questions automatically from data.

2.1 The idea in four steps

- **Enumerate** candidate questions: every feature with every possible threshold.
- **Score** each question by how **pure** the two resulting groups become.
- **Split** on the best question, then **repeat** inside each group (recursive partitioning).
- **Stop** when a group is pure, too small or deep enough; the leaf gives the answer.

2.2 Three mental pictures

- **Purity:** a node containing only "approved" customers is pure; a 50/50 node is maximally mixed. Good questions create purer children.
- **Rectangles:** every split is a straight cut parallel to an axis, so the tree carves the feature space into rectangles with one prediction each.
- **Greedy:** each step picks the best split right now without looking ahead; this is fast but not guaranteed globally optimal.

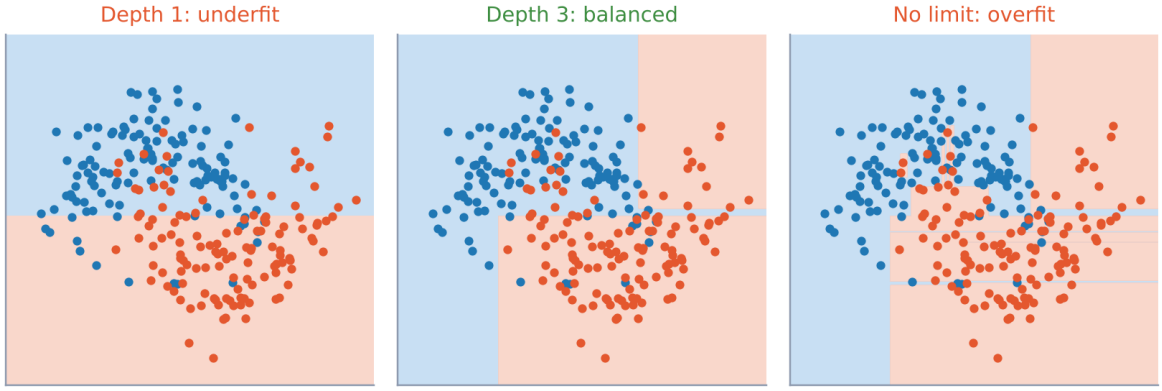


Figure 2. Depth controls complexity: depth 1 underfits, depth 3 fits well, an unlimited tree memorizes noise.

> **Key insight: bias and variance**
>
> A **shallow** tree has high bias (too simple) and low variance. A **deep** tree has low bias but high variance: it chases noise and changes drastically with new data. Depth, minimum leaf size and pruning strength balance the two. Ensembles (Section 9) cut variance by averaging many trees.

PYTHON Measuring purity in Python

```python
import numpy as np
def entropy(counts):
    p = np.array(counts) / sum(counts)
    p = p[p > 0]
    return float(-(p * np.log2(p)).sum()) + 0.0
def gini(counts):
    p = np.array(counts) / sum(counts)
    return 1 - (p ** 2).sum()
for counts in ([10, 0], [9, 1], [5, 5]):
    print(counts, 'entropy =', round(entropy(counts), 3), ' gini =', round(gini(counts), 3))
```

OUTPUT

```
[10, 0] entropy = 0.0  gini = 0.0
[9, 1] entropy = 0.469  gini = 0.18
[5, 5] entropy = 1.0  gini = 0.5
```

Code walkthrough

Lines	What the code does and why
1–5	Entropy of a node from its class counts: −Σ p·log₂(p). The line `p[p > 0]` avoids log(0).
6–8	Gini impurity: 1 − Σ p². It is the probability that two random samples from the node belong to different classes.
9–10	A pure node [10, 0] scores 0; the 50/50 node [5, 5] scores the maximum (entropy 1, Gini 0.5). Lower impurity means a purer node.

3. Mathematics

3.1 Impurity measures for classification

$$\text{Entropy}(t) = -\sum_{k=1}^{K} p_k \log_2 p_k \qquad \text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2 \qquad \text{Error}(t) = 1 - \max_k p_k$$

p_k is the fraction of class k in node t. All three equal 0 for a pure node and are largest when classes are equally mixed. Gini and entropy are smooth and usually give nearly identical trees; Gini is slightly faster because it needs no logarithm.

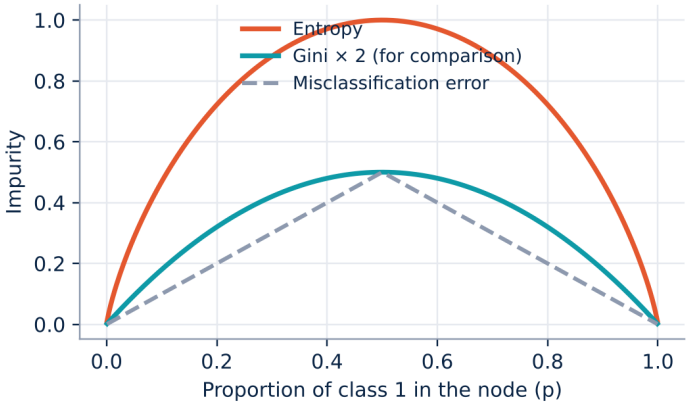


Figure 3. Impurity of a two-class node as the class proportion changes.

3.2 Information gain of a split

$$\text{Gain}(t, s) = I(t) - \sum_{j \in \{L, R\}} \frac{n_j}{n_t} I(t_j)$$

At every node the tree evaluates all features and thresholds s and picks the split with the **largest gain** (largest impurity decrease). With entropy as I this is **information gain**; with Gini it is **Gini gain**.

Worked example

Parent node: 14 samples (9 yes, 5 no). A split creates Left = 6 samples (6 yes, 0 no) and Right = 8 samples (3 yes, 5 no).

Quantity	Calculation	Value
Entropy(parent)	$-(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$	0.940
Entropy(left)	pure node	0.000
Entropy(right)	$-(3/8)\log_2(3/8) - (5/8)\log_2(5/8)$	0.954
Weighted child entropy	(6/14)(0) + (8/14)(0.954)	0.545
Information gain	0.940 − 0.545	**0.395**
Gini gain	0.459 − (8/14)(0.469)	**0.191**

3.3 Other classification criteria

Criterion	Idea	Used by
Information gain	Entropy reduction; favours features with many values	ID3
Gain ratio	Information gain divided by the split information (penalizes many-valued features)	C4.5, C5.0
Gini gain	Gini impurity reduction, binary splits	CART (scikit-learn)
Chi-square	Statistical test of independence between feature and class; multiway splits	CHAID
Log loss	Entropy of the node's class distribution	DecisionTreeClassifier(criterion='log_loss')

3.4 Regression trees: variance reduction

$$\text{SSE}(t) = \sum_{i \in t} (y_i - \bar{y}_t)^2 \qquad \Delta = \text{SSE}(t) - \text{SSE}(t_L) - \text{SSE}(t_R)$$

The leaf prediction is the **mean** of its targets (the value that minimizes squared error), and the chosen split is the one that reduces the sum of squared errors most. Example: y = 1, 2, 3, 10, 11, 12. The parent SSE is 125.5; splitting between 3 and 10 gives SSE = 2 + 2 = 4, a reduction of **121.5**.

$$\hat{y}(\mathbf{x}) = \text{mode}\{y_i : i \in \text{leaf}(\mathbf{x})\} \quad \text{(classification)} \qquad \hat{y}(\mathbf{x}) = \frac{1}{|\text{leaf}|} \sum_{i \in \text{leaf}(\mathbf{x})} y_i \quad \text{(regression)}$$

3.5 The training algorithm (CART)

Step	Action
1	Start with all training rows at the root.
2	For every feature, sort its values and test each midpoint between consecutive distinct values as a threshold.
3	Compute the impurity decrease of each candidate split; keep the best (feature, threshold).
4	Partition the rows into left (≤ threshold) and right (> threshold) and recurse on each child.
5	Stop when the node is pure, depth or size limits are reached, or no split improves impurity; make it a leaf.
6	Optionally prune the full tree back using cost-complexity pruning.

Complexity: sorting one feature costs O(n log n), so building the tree costs about O(p · n log n), and a prediction costs only O(depth).

3.6 Stopping rules and pruning

Type	How	scikit-learn parameters
Pre-pruning (early stopping)	Stop growing when a limit is hit	max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, min_impurity_decrease
Post-pruning	Grow a full tree, then cut weak branches by cost-complexity	ccp_alpha

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

R(T) is the tree's training error, $|T|$ its number of leaves and α the price per leaf. Increasing α removes the branches that buy the least accuracy per leaf. Choose α by cross-validation.

3.7 Feature importance

$$\mathrm{Imp}(f) = \sum_{t\,:\,\text{split on } f} \frac{n_t}{n}\,\Delta I(t) \qquad \text{(normalized to sum to 1)}$$

Impurity-based importance adds up the weighted impurity decrease of every split that uses the feature. It is fast but biased toward features with many distinct values; confirm it with **permutation importance** (Section 7).

PYTHON A decision tree from scratch (Gini, binary splits)

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
Xa, ya = load_breast_cancer(return_X_y=True)
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(Xa, ya, test_size=0.25, random_state=42,
    stratify=ya)
def gini_y(y):
    p = np.bincount(y) / len(y)
    return 1 - (p ** 2).sum()
def best_split(X, y, min_leaf=5):
    best, parent = (None, None, 0.0), gini_y(y)
    for j in range(X.shape[1]):
        vals = np.unique(X[:, j])
        for t in (vals[:-1] + vals[1:]) / 2:
            left = X[:, j] <= t
```

```
            if left.sum() < min_leaf or (~left).sum() < min_leaf: continue
            g = parent - (left.mean() * gini_y(y[left]) + (~left).mean() * gini_y(y[~left]))
            if g > best[2]: best = (j, t, g)
    return best
def build_tree(X, y, depth=0, max_depth=3):
    j, t, g = best_split(X, y)
    if depth == max_depth or j is None or len(np.unique(y)) == 1:
        return {'leaf': int(np.bincount(y, minlength=2).argmax())}
    m = X[:, j] <= t
    return {'f': j, 't': t, 'L': build_tree(X[m], y[m], depth + 1, max_depth),
            'R': build_tree(X[~m], y[~m], depth + 1, max_depth)}
def predict_one(node, x):
    while 'leaf' not in node:
        node = node['L'] if x[node['f']] <= node['t'] else node['R']
    return node['leaf']
my_tree = build_tree(Xa_tr, ya_tr, max_depth=3)
pred_scratch = np.array([predict_one(my_tree, x) for x in Xa_te])
print('from-scratch tree test accuracy:', round((pred_scratch == ya_te).mean(), 3))
print('root split: feature', my_tree['f'], '<=', round(float(my_tree['t']), 3))
from sklearn.tree import DecisionTreeClassifier
sk = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xa_tr, ya_tr)
print('scikit-learn tree test accuracy  :', round(sk.score(Xa_te, ya_te), 3))
print('scikit-learn root split: feature', sk.tree_.feature[0], '<=',
    round(float(sk.tree_.threshold[0]), 3))
```

OUTPUT

```
from-scratch tree test accuracy: 0.944
root split: feature 20 <= 16.795
scikit-learn tree test accuracy  : 0.944
scikit-learn root split: feature 20 <= 16.795
```

Code walkthrough

Lines	What the code does and why
1–4	Load the breast-cancer data as plain NumPy arrays and make the same stratified split used in later sections.
5–7	Gini impurity computed directly from a label array.
8–17	The heart of CART: for every feature, try each midpoint as a threshold, skip splits that leave too few rows, compute the Gini gain, and remember the best (feature, threshold, gain).
18–24	Recursion: find the best split, stop if the depth limit is reached or the node is pure (then store the majority class as a leaf), otherwise split the rows and build the left and right subtrees.
25–28	Prediction simply walks from the root to a leaf, following each test.
29–32	Train a depth-3 tree, predict the test rows one by one and report accuracy, plus the root question the algorithm discovered.
33–36	scikit-learn's optimized CART finds the same root question and the same accuracy (0.944), which confirms that the from-scratch version implements the same algorithm.

PYTHON Regression split search

```
def best_split_reg(x, y):
    order = np.argsort(x); x, y = x[order], y[order]
    best = (None, np.inf)
    for i in range(1, len(x)):
        t = (x[i] + x[i - 1]) / 2
        sse = ((y[:i] - y[:i].mean()) ** 2).sum() + ((y[i:] - y[i:].mean()) ** 2).sum()
        if sse < best[1]: best = (float(t), float(sse))
    return best
x = np.array([1, 2, 3, 4, 5, 6.]); y = np.array([1, 2, 3, 10, 11, 12.])
print('parent SSE          :', ((y - y.mean()) ** 2).sum())
print('best (threshold, SSE):', best_split_reg(x, y))
```

OUTPUT

```
parent SSE          : 125.5
best (threshold, SSE): (3.5, 4.0)
```

Code walkthrough

Lines	What the code does and why
1–8	Sort the feature, try every midpoint, compute the SSE of the two halves around their own means, and keep the smallest.
9–11	The best threshold is 3.5 with SSE 4, matching the hand calculation (parent SSE 125.5).

4. Evaluation Metrics

Tree models are judged with the usual classification or regression metrics, plus a few tree-specific checks. Always evaluate on **unseen data** (a test set or cross-validation): a deep tree can reach 100% accuracy on the data it was trained on.

4.1 Classification trees

	Predicted 1 (positive)	Predicted 0 (negative)
Actual 1	TP: true positive	FN: false negative
Actual 0	FP: false positive	TN: true negative

$$\text{Accuracy} = \frac{TP+TN}{n} \qquad \text{Precision} = \frac{TP}{TP+FP} \qquad \text{Recall} = \frac{TP}{TP+FN} \qquad F_1 = \frac{2PR}{P+R}$$

Metric	Meaning	Note for trees
Accuracy	Fraction of correct predictions	Misleading for imbalanced classes
Precision / Recall / F1	Quality of positive predictions vs coverage of real positives	Tune class_weight and the threshold for the costly error
ROC-AUC / PR-AUC	Ranking quality from predicted probabilities	A single tree yields coarse probabilities (leaf fractions), so AUC is lower than for ensembles
Log loss / Brier	Quality of the probabilities	Unpruned trees output 0/1 probabilities and are heavily penalized
MCC	Balanced correlation of predicted and actual labels	Good for imbalanced data

4.2 Regression trees

$$\text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \text{MSE} = \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2 \qquad \text{RMSE} = \sqrt{\text{MSE}} \qquad R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

MAE is robust and in the unit of *y*; **RMSE** punishes large errors; **R^2** is the share of variance explained and can be **negative** on test data when the model is worse than predicting the mean (common for overfitted deep trees).

4.3 Tree-specific diagnostics

Check	What it tells you
Train vs test gap	100% train but much lower test score signals overfitting
Depth and number of leaves	Simpler trees are easier to explain; compare at equal accuracy
Feature importance (MDI)	Total impurity reduction per feature; fast but biased to high-cardinality features
Permutation importance	Drop in test score when a feature is shuffled; more reliable
Cross-validation / OOB score	Trees are unstable, so use repeated estimates, not one split
Learning curve	Shows whether more data or a simpler model is needed

4.4 Computing classification metrics in Python

PYTHON Load data, fit a tree and compute every metric

```python
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print('shape:', X.shape, ' classes:', y.value_counts().to_dict())
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
tree = DecisionTreeClassifier(max_depth=4, random_state=0)
tree.fit(X_train, y_train)
pred = tree.predict(X_test)
proba = tree.predict_proba(X_test)[:, 1]
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix, matthews_corrcoef)
print(confusion_matrix(y_test, pred))
for f in (accuracy_score, precision_score, recall_score, f1_score, matthews_corrcoef):
    print(f'{f.__name__:20s}', round(f(y_test, pred), 3))
print(f'{"roc_auc_score":20s}', round(roc_auc_score(y_test, proba), 3))
print('train accuracy:', round(tree.score(X_train, y_train), 3))
print('test accuracy :', round(tree.score(X_test, y_test), 3))
```

OUTPUT

```
shape: (569, 30)  classes: {1: 357, 0: 212}
[[49  4]
 [ 3 87]]
accuracy_score       0.951
precision_score      0.956
recall_score         0.967
f1_score             0.961
matthews_corrcoef    0.895
roc_auc_score        0.943
train accuracy: 0.988
test accuracy : 0.951
```

Code walkthrough

Lines	What the code does and why
1–7	Load 569 tumours with 30 measurements as pandas objects (1 = benign, 0 = malignant). Trees need no feature scaling.
8–9	A stratified split keeps the class ratio equal in train and test.
10–13	Fit a tree limited to depth 4. `predict` returns labels; `predict_proba` returns the class fractions inside the leaf each row falls into.
14–19	Print the confusion matrix (rows = actual, columns = predicted) and loop over the metric functions. AUC needs the probabilities, not the labels.
20–21	Compare train and test accuracy to check for overfitting.

5. Use Cases

Domain	Task	Type
Banking	Approve or reject a loan; credit scorecards	Classification
Healthcare	Diagnosis and triage from symptoms and lab tests	Classification
Telecom / SaaS	Predict customer churn	Classification
Fraud detection	Flag suspicious transactions	Classification
Marketing	Segmentation, response to an offer	Classification
Manufacturing	Fault diagnosis, defect detection	Classification
Real estate	Estimate house price	Regression
Retail / Energy	Forecast demand or electricity load	Regression
Insurance	Estimate claim amount or premium	Regression
Finance	Risk grading and rule extraction	Both
Human resources	Attrition prediction, salary bands	Both
Agriculture	Crop disease class, yield estimate	Both
Cybersecurity	Anomaly detection (Isolation Forest)	Anomaly

5.1 When to use a tree

- **Explanations are required** (credit, medicine, law): every prediction is a readable path of rules.
- **Mixed, messy tabular data** with non-linear effects, interactions and missing values.
- **Quick baseline** that needs no scaling, plus the foundation of the strongest tabular models (forests and boosting).

5.2 When to avoid or upgrade

Situation	Better choice
Interpretability is not needed and accuracy matters most	Random Forest, Gradient Boosting, XGBoost, LightGBM
Strong linear relationship	Linear or logistic regression (trees approximate lines with steps)
Values must be extrapolated beyond the training range	Linear models, or trees with a linear model in the leaves
Images, audio, raw text	Deep learning
Very small noisy dataset	Regularized linear models or a heavily pruned tree

Common pitfalls

Overfitting: never deploy an unlimited tree; tune depth, leaf size or ccp_alpha with cross-validation.
Leakage: an ID-like or future-information feature can make a tree look perfect.
Importance misreading: impurity importance favours continuous or high-cardinality features.
Instability: explain a single tree carefully, because a small data change can alter its structure.

6. Python Code Structure for Tree Projects

scikit-learn trees follow the same small estimator interface as every other model, but they also expose the **internal tree structure**, which lets you inspect and explain every decision. Understanding this structure makes your code clean, explainable and leak-free.

6.1 The estimator API for trees

Step / member	What it does	Example
Instantiate	Create the model and set hyper-parameters (nothing learned yet)	`m = DecisionTreeClassifier(max_depth=4)`
.fit(X, y)	Grow the tree on training data; returns the model	`m.fit(X_train, y_train)`
.predict(X)	Class labels (or numbers for regressors)	`m.predict(X_test)`
.predict_proba(X)	Class fractions in the leaf (classifiers)	`m.predict_proba(X_test)`
.score(X, y)	Accuracy (classifier) or R^2 (regressor)	`m.score(X_test, y_test)`
.apply(X)	Index of the leaf each row lands in	`m.apply(X_test)`
.decision_path(X)	Sparse matrix of the nodes each row passes through	`m.decision_path(X_test)`
.feature_importances_	Impurity-based importance per feature	`m.feature_importances_`
.get_depth() / .get_n_leaves()	Size of the fitted tree	`m.get_depth()`
.tree_	Low-level arrays: children_left, children_right, feature, threshold, value, n_node_samples	`m.tree_.threshold`

6.2 Key hyper-parameters

Parameter	Meaning	Effect / typical choice
criterion	gini, entropy, log_loss (classifier); squared_error, friedman_mse, absolute_error, poisson (regressor)	Gini/squared_error by default; rarely changes accuracy much
max_depth	Maximum depth	Main complexity control: try 3–10
min_samples_split	Minimum rows needed to split a node	Larger = simpler tree
min_samples_leaf	Minimum rows in every leaf	Larger = smoother, more stable; 5–50 is common
max_leaf_nodes	Cap on the number of leaves (best-first growth)	Direct size budget
max_features	Features considered at each split	Adds randomness; used inside forests
min_impurity_decrease	Minimum gain required to split	Stops weak splits
ccp_alpha	Cost-complexity pruning strength	Choose by cross-validation
class_weight	Reweights classes (classifier)	'balanced' for imbalanced data
random_state	Seed for tie-breaking and feature shuffling	Set it for reproducibility

6.3 Inspecting the tree structure

PYTHON Tree internals: arrays, decision paths and leaves

```python
clf = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
t = clf.tree_
print('nodes:', t.node_count, ' depth:', t.max_depth, ' leaves:', clf.get_n_leaves())
print(pd.DataFrame({'left': t.children_left, 'right': t.children_right,
    'feature': t.feature, 'threshold': t.threshold.round(2),
    'samples': t.n_node_samples}).head(7))
row = X_test.iloc[[0]]
leaf = clf.apply(row)[0]
for node in clf.decision_path(row).indices:
    if node == leaf:
        print(f'leaf {node}: predict class {clf.predict(row)[0]}')
        continue
    f, th = t.feature[node], t.threshold[node]
    val = row.iloc[0, f]
    print(f'node {node}: {X.columns[f]} = {val:.3f} {"<=" if val <= th else ">"} {th:.3f}')
```

OUTPUT

```
nodes: 13  depth: 3  leaves: 7
   left  right  feature  threshold  samples
0     1      8       20      16.80      426
1     2      5       27       0.14      284
2     3      4       10       1.05      252
3    -1     -1       -2      -2.00      251
4    -1     -1       -2      -2.00        1
5     6      7       21      25.62       32
6    -1     -1       -2      -2.00       12
node 0: worst radius = 14.450 <= 16.795
node 1: worst concave points = 0.080 <= 0.136
node 2: radius error = 0.383 <= 1.048
leaf 3: predict class 1
```

Code walkthrough

Lines	What the code does and why
1–6	A fitted tree is stored in parallel arrays: for node i, the children indices, the feature index, the threshold and the number of training samples. A feature of −2 marks a leaf (and children of −1).
7–15	Explain one prediction: `decision_path` lists the nodes the row visits, and `apply` gives its final leaf. Each printed line is one question and its answer, exactly the kind of audit trail regulators ask for.

6.4 Preprocessing: categorical features and missing values

Trees need **no scaling** and, since scikit-learn 1.3, **handle missing values natively** in `DecisionTreeClassifier/Regressor` and the forests. Categorical text columns must still be encoded: one-hot encoding is safest for nominal categories, ordinal encoding is acceptable for high-cardinality ones. Always place the encoder inside a **Pipeline** so it is fitted on training data only.

PYTHON Pipeline with a categorical column and missing values

```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
df = pd.DataFrame({'age': [25, 40, 35, 52, 23, 44, 61, 30],
    'city': list('ABACBCAB'), 'income': [30, 80, np.nan, 90, 25, 70, 55, 40],
    'bought': [0, 1, 0, 1, 0, 1, 1, 0]})
pre = ColumnTransformer([('city', OneHotEncoder(handle_unknown='ignore',
    sparse_output=False), ['city'])],
                        remainder='passthrough')
pipe = make_pipeline(pre, DecisionTreeClassifier(max_depth=2, random_state=0))
pipe.fit(df.drop(columns='bought'), df['bought'])
print(list(pipe[0].get_feature_names_out()))
print('predictions:', pipe.predict(df.drop(columns='bought')))
```

OUTPUT

```
['city__city_A', 'city__city_B', 'city__city_C', 'remainder__age', 'remainder__income']
predictions: [0 1 0 1 0 1 1 0]
```

Code walkthrough

Lines	What the code does and why
1–6	A small customer table with a text column (city) and a missing income value (NaN).
7–10	The ColumnTransformer one-hot encodes *city* and passes numeric columns through untouched (including the NaN). The tree handles the NaN by itself. Wrapping both steps in a pipeline keeps preprocessing and model together.
11–12	`get_feature_names_out` shows the final column names the tree sees; the predictions run end-to-end from raw data.

6.5 Recommended project workflow

#	Stage	Python tool	Notes
1	Load and inspect	pandas: info, describe, value_counts	Check class balance, leakage columns, missing values
2	Split first	train_test_split(stratify=y)	Split before fitting anything
3	Preprocess	ColumnTransformer + Pipeline	Encode categoricals only; scaling not needed
4	Fit and tune	GridSearchCV / RandomizedSearchCV	Tune depth, min_samples_leaf, ccp_alpha
5	Evaluate	Metrics, permutation importance, learning curve	Touch the test set once
6	Explain and deploy	export_text, plot_tree, joblib.dump	Save the whole pipeline

6.6 A reusable evaluation helper

PYTHON One function to compare any tree-based classifier

```python
from sklearn.metrics import log_loss
def evaluate(name, m):
    m.fit(X_train, y_train)
    pr = m.predict_proba(X_test)[:, 1]
    lab = (pr >= 0.5).astype(int)
    return {'model': name, 'acc': accuracy_score(y_test, lab), 'f1': f1_score(y_test, lab),
            'auc': roc_auc_score(y_test, pr), 'train_acc': m.score(X_train, y_train)}
from sklearn.ensemble import RandomForestClassifier
rows = [evaluate('Tree (unlimited)', DecisionTreeClassifier(random_state=0)),
        evaluate('Tree (depth 4)', DecisionTreeClassifier(max_depth=4, random_state=0)),
        evaluate('Random forest', RandomForestClassifier(300, random_state=0))]
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

OUTPUT

```
           model   acc    f1   auc  train_acc
Tree (unlimited) 0.937 0.950 0.934      1.000
  Tree (depth 4) 0.951 0.961 0.943      0.988
   Random forest 0.958 0.967 0.995      1.000
```

Code walkthrough

Lines	What the code does and why
1–7	The helper fits any classifier with `predict_proba` and returns test metrics plus the training accuracy, which exposes overfitting at a glance.
8–12	Because every scikit-learn estimator shares `fit` and `predict_proba`, swapping a tree for a forest needs only one new line. The unlimited tree shows train accuracy 1.0 with lower test scores.

Suggested folder layout:

```
decision-tree-project/
|-- data/            raw and processed datasets
|-- notebooks/       01_eda.ipynb, 02_modeling.ipynb
|-- src/
|   |-- features.py  cleaning, encoding, feature engineering
|   |-- train.py     builds the Pipeline, tunes with CV, saves the model
|   `-- explain.py   export_text, plot_tree, permutation importance
|-- models/          saved pipelines (joblib)
|-- requirements.txt pinned library versions
`-- README.md        problem, results, how to run
```

7. Classification Trees

DecisionTreeClassifier grows a CART tree with Gini impurity (default), entropy or log loss. Each leaf predicts the **majority class** of its training rows and reports the class fractions as probabilities. The tree below was grown to depth 3 on the breast-cancer data: every box shows the question, the impurity, the number of samples, the class counts and the predicted class (colour).

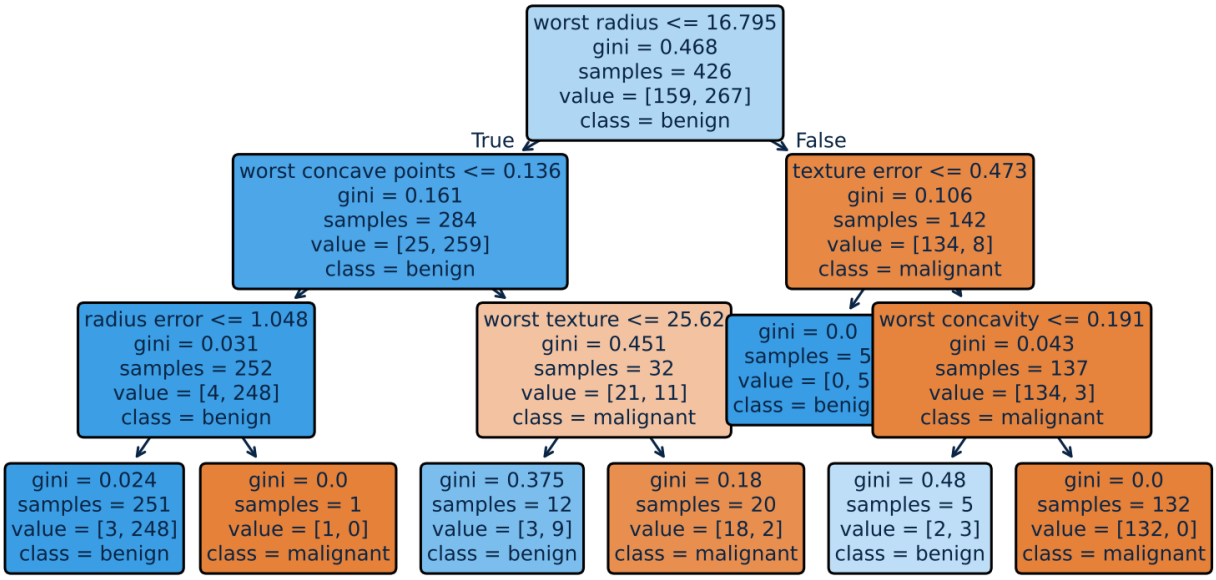


Figure 4. A real depth-3 classification tree (breast-cancer data).

7.1 Reading the tree as IF-THEN rules

PYTHON Export the tree as rules

```python
from sklearn.tree import export_text
clf = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
print(export_text(clf, feature_names=list(X.columns)))
print('depth:', clf.get_depth(), ' leaves:', clf.get_n_leaves())
print('probabilities of first test row:', clf.predict_proba(X_test.iloc[:1]).round(3))
```

OUTPUT

```
|--- worst radius <= 16.80
|   |--- worst concave points <= 0.14
|   |   |--- radius error <= 1.05
|   |   |   |--- class: 1
|   |   |--- radius error >  1.05
|   |   |   |--- class: 0
|   |--- worst concave points >  0.14
|   |   |--- worst texture <= 25.62
|   |   |   |--- class: 1
|   |   |--- worst texture >  25.62
|   |   |   |--- class: 0
|--- worst radius >  16.80
|   |--- texture error <= 0.47
|   |   |--- class: 1
|   |--- texture error >  0.47
|   |   |--- worst concavity <= 0.19
|   |   |   |--- class: 1
|   |   |--- worst concavity >  0.19
|   |   |   |--- class: 0
```

```
depth: 3  leaves: 7
probabilities of first test row: [[0.012 0.988]]
```

Code walkthrough

Lines	What the code does and why
1–3	`export_text` prints the tree as nested rules. Indentation shows depth, each line is a test and the lines ending in `class: 1` are leaves. This makes trees **white-box** models that can be pasted into documentation or business rules.
4–5	Report the tree size and the probabilities for one row. These are just the class fractions of the leaf, which is why a single tree gives very coarse probabilities.

7.2 Splitting criteria

PYTHON Gini vs entropy vs log loss

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(5, shuffle=True, random_state=42)
for crit in ('gini', 'entropy', 'log_loss'):
    m = DecisionTreeClassifier(criterion=crit, max_depth=4, random_state=0)
    print(f'{crit:9s} CV accuracy = {cross_val_score(m, X_train, y_train,
        cv=cv).mean():.3f}')
```

OUTPUT

```
gini      CV accuracy = 0.932
entropy   CV accuracy = 0.925
log_loss  CV accuracy = 0.925
```

Code walkthrough

Lines	What the code does and why
1–5	Compare the three criteria with identical depth limits and stratified 5-fold cross-validation. `entropy` and `log_loss` are the same criterion. Differences are small and depend on the data: pick by cross-validation, not by theory.

7.3 Overfitting and the depth limit

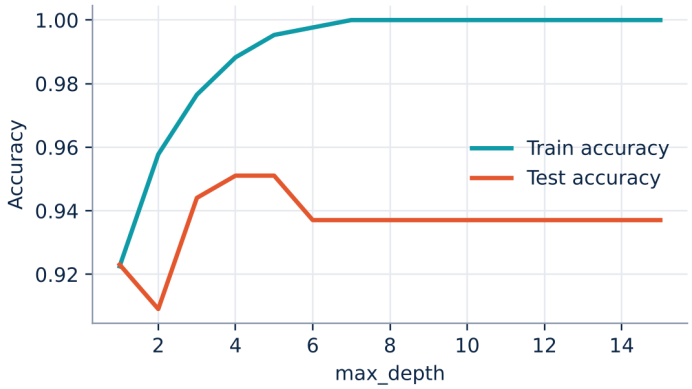


Figure 5. Training accuracy keeps rising to 100% while test accuracy stalls and then falls.

PYTHON Effect of max_depth

```python
for d in (1, 2, 3, 5, 8, None):
    m = DecisionTreeClassifier(max_depth=d, random_state=0).fit(X_train, y_train)
    print(f'max_depth={str(d):4s} leaves={m.get_n_leaves():3d}  train={m.score(X_train,
        y_train):.3f}  test={m.score(X_test, y_test):.3f}')
```

OUTPUT

```
max_depth=1    leaves=  2  train=0.923  test=0.923
max_depth=2    leaves=  4  train=0.958  test=0.909
max_depth=3    leaves=  7  train=0.977  test=0.944
max_depth=5    leaves= 15  train=0.995  test=0.951
max_depth=8    leaves= 18  train=1.000  test=0.937
max_depth=None leaves= 18  train=1.000  test=0.937
```

Code walkthrough

Lines	What the code does and why
1–3	Train trees of increasing depth (`None` = unlimited). The number of leaves grows, training accuracy reaches 1.000, but test accuracy peaks near depth 5 and then drops: the tree memorizes noise.

7.4 Tuning and cost-complexity pruning

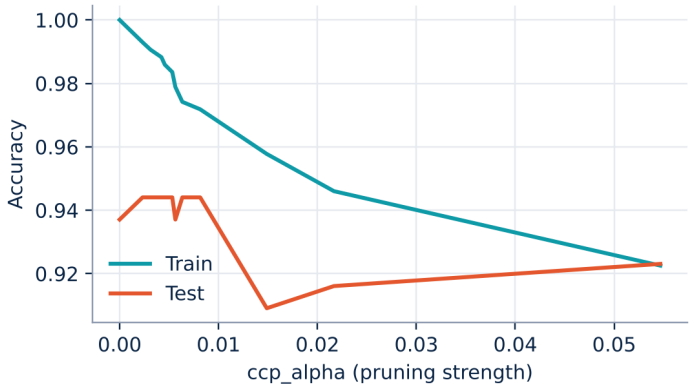


Figure 6. Larger ccp_alpha prunes the tree; the best test accuracy lies at a moderate alpha.

PYTHON GridSearchCV over depth, leaf size and pruning

```python
from sklearn.model_selection import GridSearchCV
param = {'max_depth': [2, 3, 4, 5, None],
         'min_samples_leaf': [1, 5, 10],
         'ccp_alpha': [0, 0.005, 0.01, 0.02]}
grid = GridSearchCV(DecisionTreeClassifier(random_state=0), param, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print(grid.best_params_)
print('best CV F1:', round(grid.best_score_, 3))
best = grid.best_estimator_
print('leaves:', best.get_n_leaves(), ' depth:', best.get_depth())
print('test accuracy:', round(best.score(X_test, y_test), 3))
```

OUTPUT

```
{'ccp_alpha': 0.01, 'max_depth': 3, 'min_samples_leaf': 1}
best CV F1: 0.95
leaves: 5  depth: 3
test accuracy: 0.944
```

Code walkthrough

Lines	What the code does and why
1–4	Define a grid over depth, minimum leaf size and pruning strength. Each combination is a different tree complexity.
5–8	GridSearchCV tries all 60 combinations with 5-fold cross-validation and keeps the best F1 score.
9–11	`best_estimator_` is already refitted on all training data. The tuned tree is small and generalizes well.

7.5 Feature importance

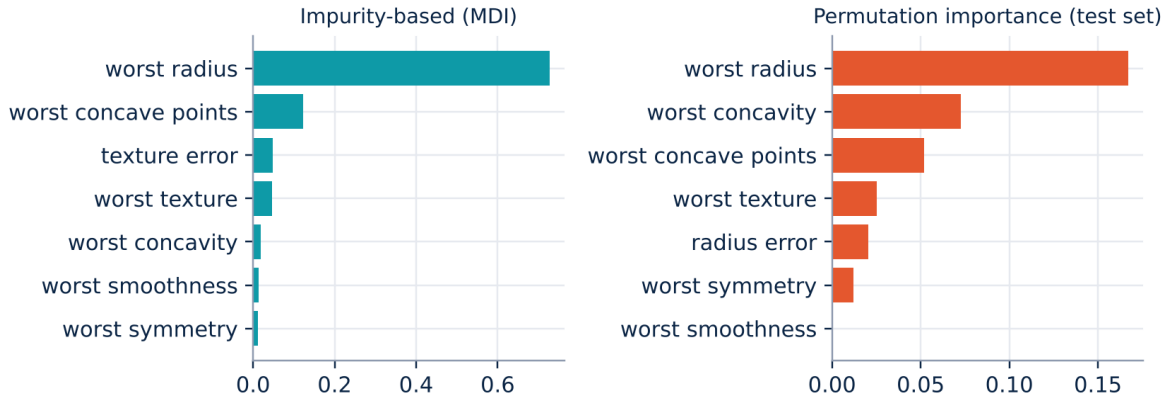


Figure 7. Impurity-based importance (left) and permutation importance on the test set (right).

PYTHON MDI vs permutation importance

```python
imp = pd.Series(tree.feature_importances_, index=X.columns)
print(imp.sort_values(ascending=False).head(5).round(3))
from sklearn.inspection import permutation_importance
pi = permutation_importance(tree, X_test, y_test, n_repeats=10, random_state=0)
print(pd.Series(pi.importances_mean,
    index=X.columns).sort_values(ascending=False).head(4).round(3))
```

OUTPUT

```
worst radius            0.727
worst concave points    0.122
texture error           0.048
worst texture           0.046
worst concavity         0.018
dtype: float64
worst radius            0.167
worst concavity         0.073
worst concave points    0.052
worst texture           0.025
dtype: float64
```

Code walkthrough

Lines	What the code does and why
1–2	`feature_importances_` sums the impurity reduction each feature produced, normalized to 1. Features never used in a split score 0.
3–5	Permutation importance shuffles one feature at a time on the **test** set and measures the drop in accuracy: model-agnostic and less biased than MDI.

7.6 Imbalanced classes

PYTHON class_weight on a rare class

```python
rng = np.random.RandomState(0)
idx0 = np.where(y_train == 0)[0]; idx1 = np.where(y_train == 1)[0]
keep = np.r_[rng.choice(idx0, 20, replace=False), idx1]
Xi, yi = X_train.iloc[keep], y_train.iloc[keep]
print('imbalanced counts:', yi.value_counts().to_dict())
from sklearn.metrics import recall_score
for cw in [None, 'balanced']:
    m = DecisionTreeClassifier(max_depth=4, class_weight=cw, random_state=0).fit(Xi, yi)
    print(f'class_weight={str(cw):9s} recall(malignant) = {recall_score(y_test,
        m.predict(X_test), pos_label=0):.3f}')
```

OUTPUT

```
imbalanced counts: {1: 267, 0: 20}
class_weight=None      recall(malignant) = 0.830
class_weight=balanced  recall(malignant) = 0.811
```

Code walkthrough

Lines	What the code does and why
1–5	Keep only 20 malignant (class 0) rows to simulate a rare class.
6–9	Compare recall on the rare class (`pos_label=0`). `class_weight='balanced'` reweights the impurity so the rare class counts more, but here it does **not** help (0.811 vs 0.830): with only 20 rare rows a single tree has little to learn from, and weights cannot create information. Combine weights with more data or resampling, an ensemble, threshold tuning and PR-AUC.

8. Regression Trees

DecisionTreeRegressor splits to minimize squared error, and each leaf predicts the **mean** of its training targets. The fitted function is therefore a **staircase** of constant steps; a tree with *L* leaves can output only *L* distinct values.

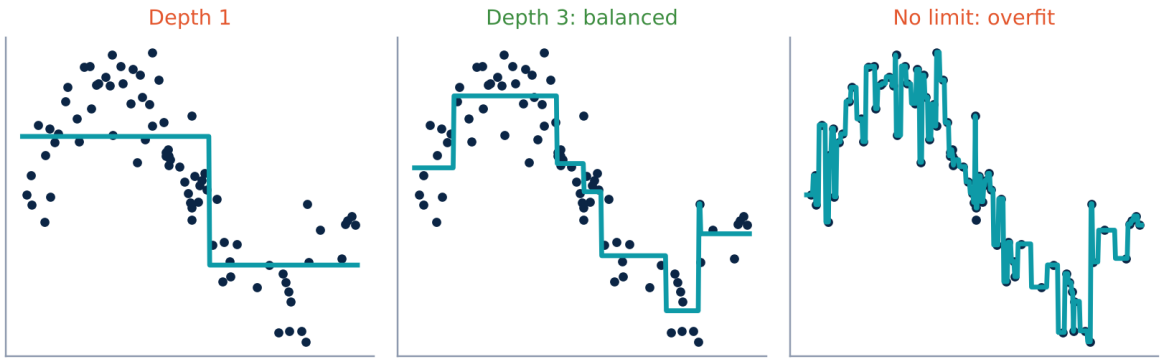


Figure 8. A regression tree approximates a smooth curve by steps; depth controls the resolution.

criterion	Minimizes	Leaf value	Use when
squared_error	MSE / variance (default)	Mean	General purpose
friedman_mse	MSE with an improvement score	Mean	Boosting
absolute_error	MAE	Median	Outliers in the target (slower)
poisson	Poisson deviance	Mean	Count targets

8.1 A staircase fit

PYTHON Fit a noisy sine curve

```python
from sklearn.tree import DecisionTreeRegressor
rng = np.random.RandomState(2)
xs = np.sort(rng.uniform(0, 6, 80))[:, None]
ys = np.sin(xs[:, 0]) + rng.normal(0, 0.25, 80)
reg = DecisionTreeRegressor(max_depth=3, random_state=0).fit(xs, ys)
grid_x = np.linspace(0, 6, 400)[:, None]
print('leaves:', reg.get_n_leaves())
print('distinct predictions:', len(np.unique(reg.predict(grid_x))))
print(export_text(reg, feature_names=['x'], decimals=2))
```

OUTPUT

```
leaves: 8
distinct predictions: 8
|--- x <= 3.35
|   |--- x <= 2.54
|   |   |--- x <= 0.70
|   |   |   |--- value: [0.25]
|   |   |--- x >  0.70
|   |   |   |--- value: [0.95]
|   |--- x >  2.54
|   |   |--- x <= 3.03
|   |   |   |--- value: [0.29]
|   |   |--- x >  3.03
|   |   |   |--- value: [0.01]
|--- x >  3.35
|   |--- x <= 5.09
|   |   |--- x <= 4.51
|   |   |   |--- value: [-0.61]
|   |   |--- x >  4.51
|   |   |   |--- value: [-1.15]
|   |--- x >  5.09
...
```

Code walkthrough

Lines	What the code does and why
1–4	Create noisy sine data: a smooth non-linear relationship.
5–8	A depth-3 tree has at most 8 leaves, so it outputs at most 8 different values: a step function.
9	The rules show the thresholds on x and the mean value stored in each leaf.

8.2 Regression metrics on real data

PYTHON Diabetes progression: tree vs linear regression

```python
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
Xd, yd = load_diabetes(return_X_y=True, as_frame=True)
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(Xd, yd, test_size=0.2, random_state=42)
def report(name, m):
    p = m.fit(Xd_tr, yd_tr).predict(Xd_te)
    print(f'{name:20s} MAE={mean_absolute_error(yd_te, p):6.2f}
        RMSE={mean_squared_error(yd_te, p) ** .5:6.2f}  R2={r2_score(yd_te, p):6.3f}')
report('Tree (no limit)', DecisionTreeRegressor(random_state=0))
report('Tree (depth 3)', DecisionTreeRegressor(max_depth=3, random_state=0))
report('Tree (leaf >= 20)', DecisionTreeRegressor(min_samples_leaf=20, random_state=0))
report('Linear regression', LinearRegression())
```

OUTPUT

```
Tree (no limit)      MAE= 56.57  RMSE= 72.90  R2=-0.003
Tree (depth 3)       MAE= 48.10  RMSE= 59.60  R2= 0.329
Tree (leaf >= 20)    MAE= 45.52  RMSE= 57.09  R2= 0.385
```

```
Linear regression    MAE= 42.79  RMSE= 53.85  R2= 0.453
```

Code walkthrough

Lines	What the code does and why
1–5	Load 442 patients with 10 features; the target is disease progression after one year. Split 80/20.
6–8	A helper that fits a model and prints MAE, RMSE and R^2 on the test set.
9–12	The unlimited tree memorizes the training data and its R^2 is about zero on test data. Limiting depth or leaf size helps a lot. On this almost linear dataset, plain linear regression still wins.

8.3 Criteria, tuning and the extrapolation limit

PYTHON Compare criteria and tune with cross-validation

```python
for crit in ('squared_error', 'friedman_mse', 'absolute_error', 'poisson'):
    m = DecisionTreeRegressor(criterion=crit, min_samples_leaf=20, random_state=0)
    print(f'{crit:15s} test R2 = {m.fit(Xd_tr, yd_tr).score(Xd_te, yd_te):.3f}')
g = GridSearchCV(DecisionTreeRegressor(random_state=0),
    {'max_depth': [2, 3, 4, 5, 6], 'min_samples_leaf': [5, 10, 20, 30]}, cv=5,
        scoring='r2').fit(Xd_tr, yd_tr)
print(g.best_params_, ' CV R2:', round(g.best_score_, 3), ' test R2:', round(g.score(Xd_te,
    yd_te), 3))
```

OUTPUT

```
squared_error   test R2 = 0.385
friedman_mse    test R2 = 0.385
absolute_error  test R2 = 0.339
poisson         test R2 = 0.372
{'max_depth': 2, 'min_samples_leaf': 5}  CV R2: 0.352  test R2: 0.295
```

Code walkthrough

Lines	What the code does and why
1–3	Compare the four splitting criteria under the same size limit. `squared_error` and `friedman_mse` tie; `absolute_error` is more robust to outliers but slower and a little worse here.
4–6	Tune depth and leaf size with 5-fold cross-validated R^2. A very shallow tree (depth 2) wins in cross-validation, yet its test R^2 (0.295) is below the leaf-size-20 tree above (0.385): one small test set of 89 rows is a noisy judge, which is why repeated cross-validation or ensembles are preferred.

PYTHON Trees cannot extrapolate

```python
xl = np.arange(0, 5, 0.1)[:, None]; yl = 2 * xl[:, 0] + 1
t_ = DecisionTreeRegressor().fit(xl, yl); lr = LinearRegression().fit(xl, yl)
print('true y at x = 8   :', 17)
print('tree  predicts    :', round(float(t_.predict([[8]])[0]), 2))
print('linear predicts   :', round(float(lr.predict([[8]])[0]), 2))
```

OUTPUT

```
true y at x = 8   : 17
tree  predicts    : 10.8
linear predicts   : 17.0
```

Code walkthrough

Lines	What the code does and why
1–5	Train on x between 0 and 5 and predict at x = 8. The tree can only repeat the value of its last leaf (about 10.8), while linear regression extrapolates correctly. Never rely on a tree outside the range of the training data.

8.4 Multi-output regression trees

A single tree can predict **several targets at once**: each leaf stores one mean per target and splits reduce the combined error. This is cheaper and often more coherent than training one model per target.

PYTHON One tree, two targets

```python
Y2 = np.c_[np.sin(xs[:, 0]), np.cos(xs[:, 0])] + rng.normal(0, 0.1, (80, 2))
mo = DecisionTreeRegressor(max_depth=4, random_state=0).fit(xs, Y2)
print('prediction shape:', mo.predict(xs[:3]).shape)
print('R2 (average over targets):', round(mo.score(xs, Y2), 3))
```

OUTPUT

```
prediction shape: (3, 2)
R2 (average over targets): 0.982
```

Code walkthrough

Lines	What the code does and why
1–4	Create two related targets (sine and cosine) and fit one tree. `predict` returns an array with one column per target, and `score` averages R^2 over them.

9. More Tree Models: Ensembles and Variants

A single tree is easy to explain but unstable. The most accurate models for tabular data **combine many trees**. Averaging methods (bagging, forests) mainly cut **variance**; boosting methods build shallow trees one after another and mainly cut **bias**. This section explains each family with runnable code.

Problem with one tree	Remedy	Section
Too weak / simplest possible model	Decision stump, AdaBoost	9.1
High variance	Bagging, Random Forest, Extra Trees	9.2, 9.3
Needs maximum accuracy	Gradient Boosting, HistGradientBoosting, XGBoost, LightGBM	9.4, 9.5, 9.6
Need prediction intervals or monotonic effects	Quantile and monotonic boosting	9.4, 9.5
Want the best of several models	Voting, Stacking	9.7
Anomaly or outlier detection	Isolation Forest	9.8
Want tree-based features for a linear model	Random Trees Embedding	9.9
Special variants	ID3, C4.5, CHAID, oblique, model trees, survival forests	9.10

9.1 Decision stump and AdaBoost

A **decision stump** is a tree of depth 1: a single question. It is a very weak learner, but **AdaBoost** combines hundreds of stumps: each new stump focuses on the rows previous stumps misclassified by increasing their weights, and stumps vote with weights based on their accuracy.

$$F(\mathbf{x}) = \operatorname{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(\mathbf{x})\right), \qquad \alpha_m = \tfrac{1}{2}\ln\tfrac{1-\varepsilon_m}{\varepsilon_m}$$

PYTHON A single stump vs AdaBoost of stumps

```python
from sklearn.ensemble import AdaBoostClassifier
stump = DecisionTreeClassifier(max_depth=1).fit(X_train, y_train)
print('single stump test accuracy:', round(stump.score(X_test, y_test), 3))
ada = AdaBoostClassifier(n_estimators=200, random_state=0).fit(X_train, y_train)
print('AdaBoost (200 stumps)    :', round(ada.score(X_test, y_test), 3))
print('weight of first 3 stumps  :', ada.estimator_weights_[:3].round(2))
```

OUTPUT

```
single stump test accuracy: 0.909
AdaBoost (200 stumps)    : 0.972
weight of first 3 stumps  : [2.48 1.7  1.75]
```

Code walkthrough

Lines	What the code does and why
1–3	A depth-1 tree uses one feature and one threshold only, so its accuracy is limited.
4–6	AdaBoost's default base learner is a stump. Boosting 200 of them lifts accuracy a lot. `estimator_weights_` are the vote weights α_m of the stumps.

9.2 Bagging and Random Forest

Bagging (bootstrap aggregating) trains many deep trees, each on a random bootstrap sample of the rows, and averages their predictions (majority vote for classification). **Random Forest** adds a second source of randomness: each split considers only a random subset of the features (`max_features`), which decorrelates the trees and improves the average.

$$\hat{y}_{forest}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$$

Parameter	Meaning	Typical choice
n_estimators	Number of trees	200–500 (more never hurts, only costs time)
max_features	Features tried per split	sqrt(p) for classification, p/3 for regression
max_depth / min_samples_leaf	Tree size	Often unlimited; raise min_samples_leaf for noisy data
oob_score	Score rows using only trees that did not see them	True: a free cross-validation estimate
n_jobs	Parallel trees	-1 (all cores)

PYTHON Random Forest with out-of-bag score and importance

```python
rf = RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0, n_jobs=-1)
rf.fit(X_train, y_train)
print('OOB accuracy :', round(rf.oob_score_, 3))
print('test accuracy:', round(rf.score(X_test, y_test), 3))
print(pd.Series(rf.feature_importances_,
    index=X.columns).sort_values(ascending=False).head(5).round(3))
from sklearn.ensemble import BaggingClassifier
bag = BaggingClassifier(DecisionTreeClassifier(), n_estimators=100,
    random_state=0).fit(X_train, y_train)
print('bagging test accuracy:', round(bag.score(X_test, y_test), 3), ' AUC:',
    round(roc_auc_score(y_test, bag.predict_proba(X_test)[:, 1]), 3))
```

OUTPUT

```
OOB accuracy : 0.958
test accuracy: 0.958
worst concave points    0.134
worst perimeter         0.118
worst radius            0.117
mean concave points     0.110
worst area              0.108
dtype: float64
bagging test accuracy: 0.944  AUC: 0.992
```

Code walkthrough

Lines	What the code does and why
1–4	Train 300 trees on bootstrap samples. The out-of-bag score evaluates every row using only the trees that did not see it, so it estimates test performance without a separate validation set.
5	Forest importance averages the impurity decrease over all trees and is much more stable than single-tree importance: importance is spread across correlated features instead of one arbitrary pick.
6–8	Plain bagging of full trees (no feature subsampling). It already reaches a high AUC, because averaging smooths the coarse 0/1 probabilities of single trees.

9.3 Extremely Randomized Trees (Extra Trees)

Extra Trees go one step further than Random Forest: besides random feature subsets, the **split thresholds are drawn at random** and the best of those random candidates is used. Trees are built on the whole sample (no bootstrap by default). The extra randomness lowers variance further, trains faster (no sorting for the best threshold) and often gives smoother predictions.

PYTHON Extra Trees for classification and regression

```python
from sklearn.ensemble import ExtraTreesClassifier, ExtraTreesRegressor
et = ExtraTreesClassifier(300, random_state=0, n_jobs=-1).fit(X_train, y_train)
print('Extra Trees test accuracy:', round(et.score(X_test, y_test), 3), ' AUC:',
    round(roc_auc_score(y_test, et.predict_proba(X_test)[:, 1]), 3))
etr = ExtraTreesRegressor(300, random_state=0, n_jobs=-1).fit(Xd_tr, yd_tr)
print('Extra Trees regression test R2:', round(etr.score(Xd_te, yd_te), 3))
```

OUTPUT

```
Extra Trees test accuracy: 0.965  AUC: 0.996
Extra Trees regression test R2: 0.473
```

Code walkthrough

Lines	What the code does and why
1–3	The interface is identical to RandomForestClassifier, so swapping the two is a one-word change.
4–5	The regressor averages the leaf means of 300 random trees. On the diabetes data it beats a single tree by a wide margin.

9.4 Gradient Boosting

Gradient Boosting builds shallow trees **sequentially**. Each new tree is fitted to the **negative gradient of the loss** (for squared error, simply the current residuals), and its prediction is added with a small **learning rate** η. Many small, corrective steps give very accurate models.

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}), \qquad h_m \text{ fitted to } r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F = F_{m-1}}$$

Parameter	Meaning	Typical choice
n_estimators	Number of boosting rounds	100–1000; use early stopping
learning_rate (η)	Shrinks each tree's contribution	0.03–0.1; smaller needs more trees
max_depth	Depth of each weak tree	2–5
subsample	Fraction of rows per tree (stochastic boosting)	0.5–0.9
loss	squared_error, absolute_error, huber, quantile (regression); log_loss (classification)	Choose by the task

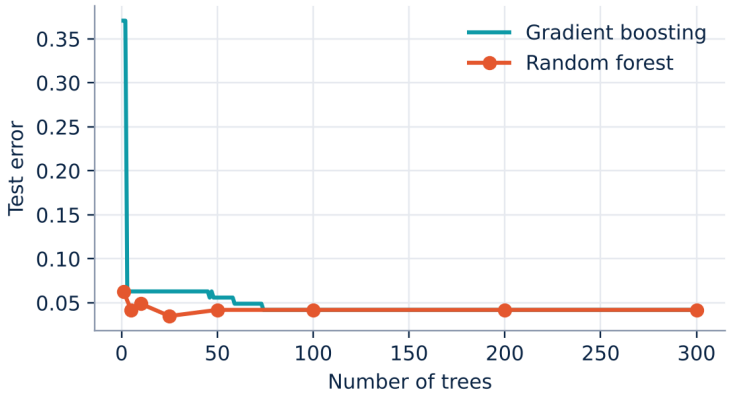


Figure 9. Test error of gradient boosting as trees are added, compared with random forests of increasing size.

PYTHON Gradient boosting with staged predictions

```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.1,
    random_state=0).fit(X_train, y_train)
err = [1 - accuracy_score(y_test, p) for p in gb.staged_predict(X_test)]
print('test error after 10 / 50 / 300 trees:', [round(err[i], 3) for i in (9, 49, 299)])
from sklearn.ensemble import GradientBoostingRegressor
lo, hi = [GradientBoostingRegressor(loss='quantile', alpha=a, max_depth=2,
    n_estimators=100, random_state=0).fit(Xd_tr, yd_tr) for a in (0.1, 0.9)]
inside = ((yd_te >= lo.predict(Xd_te)) & (yd_te <= hi.predict(Xd_te))).mean()
print('80% prediction interval covers', f'{inside:.0%}', 'of test targets')
```

OUTPUT

```
test error after 10 / 50 / 300 trees: [0.063, 0.056, 0.042]
80% prediction interval covers 80% of test targets
```

Code walkthrough

Lines	What the code does and why
1–4	`staged_predict` yields the predictions after each boosting round, so you can watch the test error fall as trees are added and choose the number of rounds.
5–8	Quantile loss turns boosting into **quantile regression**. Fitting the 10th and 90th percentiles gives an 80% prediction interval for every row; a well-calibrated interval should cover about 80% of the test targets.

9.5 HistGradientBoosting

HistGradientBoosting (scikit-learn's equivalent of LightGBM) first bins every feature into at most 255 histogram bins, which makes training **orders of magnitude faster** on large data. It also supports **missing values natively**, **categorical features**, **early stopping** and **monotonic constraints** (for example, price must not decrease as area increases).

PYTHON Histogram boosting with early stopping and a monotonic constraint

```python
from sklearn.ensemble import HistGradientBoostingClassifier, HistGradientBoostingRegressor
hgb = HistGradientBoostingClassifier(max_iter=500, learning_rate=0.05, early_stopping=True,
    validation_fraction=0.15, n_iter_no_change=20, random_state=0).fit(X_train, y_train)
print('rounds used:', hgb.n_iter_, ' test accuracy:', round(hgb.score(X_test, y_test), 3))
mono = HistGradientBoostingRegressor(monotonic_cst={'bmi': 1}, random_state=0).fit(Xd_tr,
    yd_tr)
base = Xd_te.iloc[[0]].copy(); lo_, hi_ = base.copy(), base.copy(); lo_['bmi'] -= 0.05;
    hi_['bmi'] += 0.05
print('prediction rises with bmi:', mono.predict(lo_)[0] <= mono.predict(base)[0] <=
    mono.predict(hi_)[0])
```

OUTPUT

```
rounds used: 96  test accuracy: 0.951
prediction rises with bmi: True
```

Code walkthrough

Lines	What the code does and why
1–4	Allow up to 500 rounds but stop when the validation score has not improved for 20 rounds, so the optimal number of trees is found automatically.
5–7	`monotonic_cst={'bmi': 1}` forces predicted progression never to decrease as BMI increases, injecting domain knowledge. The check lowers and raises BMI for one patient and confirms the order.

9.6 XGBoost and LightGBM

XGBoost and **LightGBM** are optimized gradient-boosting libraries. XGBoost adds L1/L2 regularization on leaf weights and a second-order (Newton) approximation of the loss. LightGBM grows trees **leaf-wise** (best leaf first) with histogram binning, making it extremely fast on big data. Both offer a scikit-learn-compatible API.

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_m \left[\gamma T_m + \tfrac{1}{2}\lambda \|w_m\|^2\right]$$

PYTHON XGBoost and LightGBM with early stopping

```python
import xgboost as xgb, lightgbm as lgb
Xtr2, Xval, ytr2, yval = train_test_split(X_train, y_train, test_size=0.2, random_state=1,
    stratify=y_train)
xm = xgb.XGBClassifier(n_estimators=1000, learning_rate=0.05, max_depth=3, subsample=0.8,
    colsample_bytree=0.8, reg_lambda=1.0, early_stopping_rounds=30, eval_metric='logloss',
        verbosity=0, random_state=0)
xm.fit(Xtr2, ytr2, eval_set=[(Xval, yval)], verbose=False)
print('XGBoost best round:', xm.best_iteration, ' test accuracy:', round(xm.score(X_test,
    y_test), 3))
lm = lgb.LGBMClassifier(n_estimators=1000, learning_rate=0.05, num_leaves=15,
    subsample=0.8, subsample_freq=1,
    colsample_bytree=0.8, verbose=-1, random_state=0)
lm.fit(Xtr2, ytr2, eval_set=[(Xval, yval)], callbacks=[lgb.early_stopping(30,
    verbose=False)])
print('LightGBM best round:', lm.best_iteration_, ' test accuracy:', round(lm.score(X_test,
    y_test), 3))
print(pd.Series(xm.feature_importances_,
    index=X.columns).sort_values(ascending=False).head(4).round(3))
```

OUTPUT

```
XGBoost best round: 340  test accuracy: 0.965
LightGBM best round: 149  test accuracy: 0.965
mean concave points     0.284
worst perimeter         0.150
worst radius            0.135
worst concave points    0.089
dtype: float32
```

Code walkthrough

Lines	What the code does and why
1–6	Hold out a validation split and let XGBoost stop when the validation log loss stops improving (`early_stopping_rounds`). `subsample`, `colsample_bytree` and `reg_lambda` regularize the model.
7–10	LightGBM is controlled by `num_leaves` (leaf-wise growth) instead of depth. Early stopping is passed as a callback.
11	Both libraries expose `feature_importances_`, which you should confirm with permutation importance or SHAP values.

9.7 Voting and Stacking ensembles

- **Voting:** averages the (soft) probabilities of several different models, for example a tree, a forest and boosting.
- **Stacking:** a meta-model (often logistic or ridge regression) learns how to combine the cross-validated predictions of the base models.

PYTHON Soft voting and stacking

```python
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
base = [('tree', DecisionTreeClassifier(max_depth=4, random_state=0)),
        ('rf', RandomForestClassifier(200, random_state=0)),
        ('gb', GradientBoostingClassifier(random_state=0))]
vote = VotingClassifier(base, voting='soft').fit(X_train, y_train)
stack = StackingClassifier(base[1:], final_estimator=LogisticRegression(max_iter=2000),
    cv=5).fit(X_train, y_train)
print('soft voting accuracy:', round(vote.score(X_test, y_test), 3))
print('stacking accuracy   :', round(stack.score(X_test, y_test), 3))
```

OUTPUT

```
soft voting accuracy: 0.944
stacking accuracy   : 0.958
```

Code walkthrough

Lines	What the code does and why
1–5	Define the base learners once: a pruned tree, a forest and gradient boosting.
6–9	Soft voting averages predicted probabilities. Stacking trains a logistic regression on the out-of-fold predictions of the base models (`cv=5` prevents leakage). Gains over the best single model are often small; here soft voting (0.944) is even below the forest alone (0.958), so always validate the combination.

9.8 Isolation Forest (anomaly detection)

Isolation Forest detects outliers rather than predicting a label. It grows random trees that split on a random feature at a random threshold. Anomalies are **few and different**, so they are isolated after very few splits (short average path length), while normal points need many splits.

$$s(\mathbf{x}, n) = 2^{-E[h(\mathbf{x})]/c(n)} \qquad (s \to 1: \text{ anomaly}, \quad s \ll 0.5: \text{ normal})$$

PYTHON Find the most anomalous tumours without using labels

```python
from sklearn.ensemble import IsolationForest
iso = IsolationForest(n_estimators=200, contamination=0.05, random_state=0).fit(X_train)
flag = iso.predict(X_test)
print('flagged as anomalies:', int((flag == -1).sum()), 'of', len(flag))
print('malignant rate among flagged :', round(float((y_test[flag == -1] == 0).mean()), 2))
print('malignant rate among the rest:', round(float((y_test[flag == 1] == 0).mean()), 2))
```

OUTPUT

```
flagged as anomalies: 5 of 143
malignant rate among flagged : 1.0
malignant rate among the rest: 0.35
```

Code walkthrough

Lines	What the code does and why
1–4	Train on the features only (no labels). `contamination` is the expected fraction of anomalies; `predict` returns −1 for anomalies and +1 for normal points.
5–6	Although labels were never used, the flagged points are far more often malignant than the rest, because malignant tumours are rarer and atypical.

9.9 Random Trees Embedding: trees as feature generators

Each leaf of a forest defines a binary region of the feature space. **RandomTreesEmbedding** (and `apply` on any forest) converts every row into a sparse vector of "which leaf did I land in" indicators. A simple linear model trained on these features can learn rich non-linear patterns; this is the idea behind Facebook's boosted-tree + logistic regression ad-ranking system.

PYTHON Leaf indicators + logistic regression

```python
from sklearn.ensemble import RandomTreesEmbedding
from sklearn.pipeline import make_pipeline
emb = make_pipeline(RandomTreesEmbedding(n_estimators=100, max_depth=4, random_state=0),
    LogisticRegression(max_iter=3000))
emb.fit(X_train, y_train)
print('embedding dimension:', emb[0].transform(X_train).shape[1], ' test accuracy:',
    round(emb.score(X_test, y_test), 3))
```

OUTPUT

```
embedding dimension: 1203  test accuracy: 0.958
```

Code walkthrough

Lines	What the code does and why
1–5	The embedding is unsupervised: 100 random trees map each row to thousands of leaf indicators, and logistic regression is trained on them. A drawback is that the number of features becomes large.

9.10 Other tree algorithms and variants

Algorithm	Split rule / idea	Notes
ID3	Information gain; categorical features only	Oldest; no pruning, no numeric support
C4.5 / C5.0	Gain ratio; numeric thresholds, missing values, post-pruning	Multiway splits on categories; C5.0 adds boosting
CART	Gini or MSE with binary splits; cost-complexity pruning	Used by scikit-learn; classification and regression
CHAID	Chi-square tests with multiway splits	Popular in marketing and survey analysis
QUEST	Unbiased feature selection via statistical tests	Removes the bias toward many-valued features
Conditional inference trees	Permutation tests decide whether and where to split	No separate pruning needed; unbiased
Oblique trees	Splits on linear combinations of features	Handle diagonal boundaries with few nodes
Model trees (M5)	A linear regression inside each leaf	Piecewise-linear; can extrapolate within leaves
Rotation Forest	PCA-rotated feature subsets for each tree	Often more accurate than Random Forest on correlated features
Random Survival Forest	Splits maximize survival-time separation	Time-to-event data with censoring
Quantile Regression Forest	Keeps all leaf values to estimate conditional quantiles	Prediction intervals from a forest
CatBoost	Ordered boosting with symmetric (oblivious) trees	Excellent native handling of categorical features
Optimal / soft trees	Globally optimized or differentiable trees	Research tools; slow but more accurate small trees

10. Model Comparison and How to Choose

All classifiers were trained on the same 426 breast-cancer rows and evaluated on the same 143 held-out rows; all regressors on the same 353 diabetes rows and tested on 89 rows. CV is 5-fold cross-validation on the training data (accuracy for classification, R^2 for regression).

10.1 Classification results

#	Model	CV acc	Test acc	F1	Test AUC
1	Extra trees	0.969	0.965	0.972	0.996
2	XGBoost	0.962	0.958	0.967	0.995
3	Random forest	0.960	0.958	0.967	0.995
4	Stacking (RF+GB -> logistic)	0.958	0.958	0.967	0.994
5	HistGradientBoosting	0.962	0.972	0.978	0.993
6	Voting (soft: tree+RF+GB)	0.946	0.944	0.956	0.993
7	Gradient boosting	0.958	0.958	0.967	0.992
8	LightGBM	0.958	0.965	0.973	0.992
9	Bagging	0.946	0.944	0.956	0.992
10	AdaBoost	0.967	0.972	0.978	0.990
11	Entropy tree (ID3/C4.5-style)	0.951	0.958	0.967	0.962
12	Pruned tree (depth 4)	0.923	0.951	0.961	0.943
13	Single tree (unpruned CART)	0.918	0.937	0.950	0.934
14	Decision stump (depth 1)	0.894	0.923	0.941	0.908

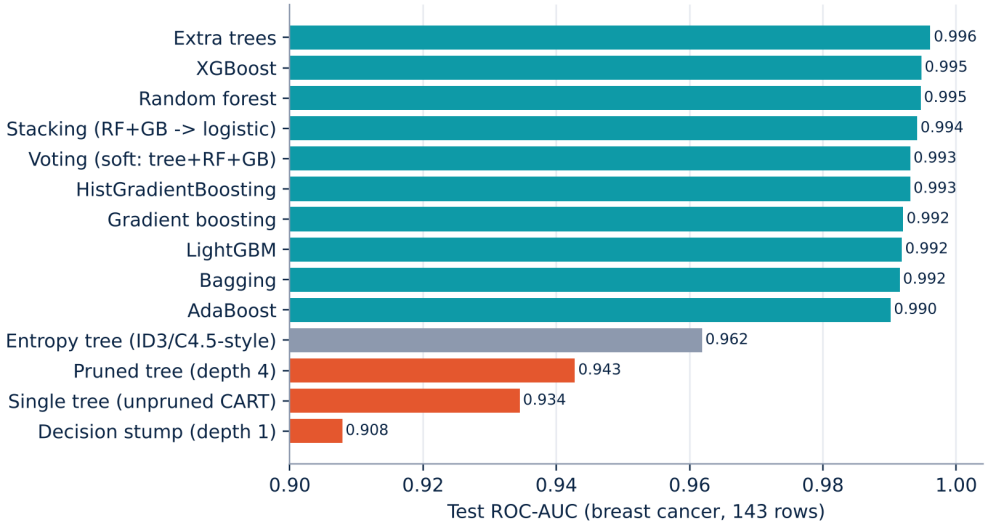


Figure 10. Test ROC-AUC of every classifier (teal = best group, orange = weakest).

What the classification results show

A **single tree** reaches 0.93–0.95 accuracy but only **0.91–0.96 AUC**, because its probabilities are coarse leaf fractions. A decision stump (one question) already gets 0.92 accuracy on this easy dataset.
Every ensemble lifts AUC to 0.99+ (bagging, forests, Extra Trees, boosting, XGBoost, LightGBM). Their accuracies (0.944–0.972) are within one or two patients of each other, so the differences are not meaningful on 143 rows. Voting and stacking add complexity without beating the best single ensemble here.

10.2 Regression results

#	Model	CV R^2	Test R^2	RMSE	MAE
1	LightGBM	0.378	0.503	51.3	40.9
2	Extra trees	0.409	0.473	52.8	43.6
3	XGBoost	0.402	0.469	53.0	42.5
4	Linear regression	0.449	0.453	53.9	42.8
5	Gradient boosting	0.363	0.451	53.9	44.7
6	AdaBoost	0.403	0.445	54.2	44.6
7	Stacking (RF+GB -> ridge)	0.391	0.439	54.5	44.3
8	Bagging	0.392	0.432	54.9	44.4
9	Random forest	0.398	0.424	55.2	44.5
10	HistGradientBoosting	0.342	0.387	57.0	45.0
11	Tree (min leaf 20)	0.277	0.385	57.1	45.5
12	Pruned tree (depth 3)	0.268	0.329	59.6	48.1
13	Single tree (unpruned)	-0.139	-0.003	72.9	56.6

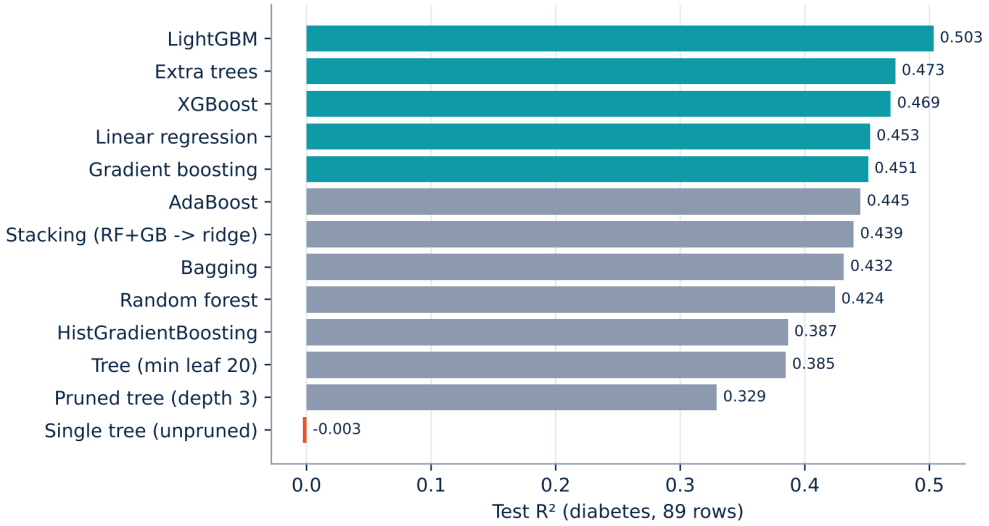


Figure 11. Test R^2 of every regressor (teal = best group, orange = weakest).

What the regression results show

The **unlimited tree** is worst (test $R^2 \approx 0$): it memorizes the training data. Pruning (depth 3 or a minimum leaf size) lifts R^2 to 0.33–0.39.

Ensembles reach roughly 0.39–0.50. The best test score (LightGBM 0.503) and **linear regression (0.453)** are close, because the diabetes data is small and almost linear. Cross-validated scores tell the same story: the CV R^2 of every ensemble (0.34–0.41) is below linear regression's 0.449.

Trees and boosting pay off when relationships are strongly non-linear or full of interactions; always benchmark against a linear model.

10.3 Decision guide

Situation	Recommended model
Explanations or business rules are required	Single pruned tree (max_depth 3–5 or ccp_alpha tuned by CV)
Fast strong baseline with little tuning	Random Forest (n_estimators 300+, oob_score=True)
Maximum accuracy on tabular data	Gradient boosting: LightGBM / XGBoost / HistGradientBoosting with early stopping
Very noisy data, want low variance	Extra Trees or Random Forest with larger min_samples_leaf
Large dataset (millions of rows)	LightGBM or HistGradientBoosting
Many categorical features	CatBoost, LightGBM, or HistGradientBoosting with categorical_features
Prediction intervals	Quantile gradient boosting, quantile regression forest
Domain constraints (effect must increase)	Monotonic constraints in HistGradientBoosting, XGBoost or LightGBM
Outliers or fraud without labels	Isolation Forest
Smooth/linear relationship or extrapolation needed	Linear or regularized models, or a model tree
Imbalanced classes	class_weight, resampling, PR-AUC, threshold tuning, boosting with scale_pos_weight

11. Cheat Sheet and Summary

Topic	Remember
Idea	Recursive yes/no questions; each split maximizes impurity decrease; leaf = majority class or mean
Classification	Gini or entropy; Gain = I(parent) − Σ (n/n) I(child); metrics: accuracy, F1, ROC-AUC
Regression	SSE / MSE reduction; leaf = mean; metrics: MAE, RMSE, R^2; cannot extrapolate
Overfitting	Limit max_depth, set min_samples_leaf, prune with ccp_alpha; tune by cross-validation
Preprocessing	No scaling; encode categoricals; missing values supported; use a Pipeline
Importance	MDI is fast but biased; confirm with permutation importance
Averaging ensembles	Bagging, Random Forest, Extra Trees: reduce variance; OOB score is a free validation

Topic	Remember
Boosting	AdaBoost, Gradient Boosting, XGBoost, LightGBM: reduce bias; use small learning rate + early stopping
Special models	Stump, Isolation Forest, Random Trees Embedding, quantile and monotonic boosting, stacking
Code habits	Stratified split first, Pipeline, tune with GridSearchCV, evaluate once on test, save the pipeline

Learning roadmap

- Re-implement a tree from scratch and compare it with scikit-learn (Section 3).
- Practise rules, depth control and pruning on a classification problem (Section 7).
- Try regression trees and observe the staircase and extrapolation limit (Section 8).
- Compare forests, boosting, XGBoost and LightGBM with cross-validation (Section 9).
- Explain models with permutation importance and decision paths (Sections 6–7).
- Package the final model as a Pipeline and save it with `joblib`.

About the author

These notes were prepared by **Chandan Chaudhari**.

GitHub: github.com/chandanc5525

Feedback, corrections and stars on the repository are always welcome.